EngageIQ Data Integration Pipeline

Complete Implementation & Architectural Verification Report

Directory: `data_pipeline/`
Status: PRODUCTION READY (19/19 TESTS PASSED)
Date: October 2026 • Windows / Python 3.13

Executive Summary: A comprehensive, standalone data integration pipeline was designed and executed inside an isolated workspace folder (`data_pipeline/`). It ingests raw event payloads from **Google Calendar**, **Slack**, **Jira**, and **GitHub**, resolves disparate employee identities into canonical company UUIDs, performs timezone normalization and noise filtering, computes the 9 target metrics, outputs the standardized unified JSON schema, evaluates dual-lens **EHS** and **OPI** scores, fuses them into the **40:60 PACE score**, stores all records in a relational SQLite database, and exposes FastAPI endpoints connected live to the React dashboard.

1 Mapping API Data Sources to Target Engagement Metrics

Target Metric	Data Source(s)	API Endpoints & Fields	Computation & Normalization Logic
meeting_load	Google Calendar	`GET /calendars/{calendarId}/events` • `start.dateTime` , `end.dateTime` • `attendees[].responseStatus`	Hours in accepted meetings over weekly working hours (e.g. 40h/week). Filters out declined meetings and all-day events. Clamped $[0.0, 1.0]$.
focus_gap	Google Calendar	`GET /calendars/{calendarId}/events` • `start` , `end` datetime intervals	Average unbroken focus blocks between meetings during employee local working hours (≥ 2 consecutive hours).
response_latency	Slack	`GET /conversations.history` • `ts` , `thread_ts` , `user`	Average duration (in minutes) between receiving a direct mention (`<@U...>`) or DM and posting a reply in the thread.
after_hours_ratio	Slack + GitHub + Jira	• Slack `ts` • GitHub `created_at` • Jira `updated`	(Activity events logged outside local work hours 9am-6pm / weekends) / Total activity events. Evaluated via employee local timezone.
completion_rate	Jira	`GET /rest/api/3/search` (JQL) • `status` , `resolutiondate`	Completed issues / Total assigned issues within sprint or time window. Clamped $[0.0, 1.0]$.
velocity	Jira	`GET /rest/api/3/issue` • `customfield_10002` (Story Points)	Total story points completed within aggregation period.
deadline_adherence	Jira	`GET /rest/api/3/search` • `duedate` , `resolutiondate`	Tasks completed on or before due date / Total completed tasks with due dates. Clamped $[0.0, 1.0]$.
code_activity	GitHub	`GET /repos/{owner}/{repo}/commits` • `GET /repos/{owner}/{repo}/pulls`	Weighted activity sum: $1.0(\text{Commits}) + 3.0(\text{PRs opened}) + 2.0(\text{PR Reviews})$. Filtered for bots.
rework_ratio	GitHub + Jira	• Jira `changelog` (reopened tickets) • GitHub reverted commits	(Reopened tickets from Done to In Progress + Reverted commits) / Total completed work items. Penalizes rework.

2 Identity Resolution & Noise Reduction Layers

Identity Resolution Table (identity_resolver.py) — STEP 1

- **Primary Key:** `employee_id` (Canonical HR UUID, e.g. `EMP-9042`).
- **Work Email:** Primary anchor connecting Calendar, Jira, and Slack accounts.
- **Slack User ID:** Maps `U01928374` to direct mention threads and DMs.
- **Jira Account ID:** Maps Atlassian account ID `5b10ac8d...` .
- **GitHub Username:** Maps `sconnor-dev` for commit/PR attribution.
- **Local Timezone:** Stores `America/New_York` , `Europe/London` , etc.

Timezone & Noise Filter (noise_filter.py) — STEP 4

- **UTC Normalization:** Normalizes heterogeneous timestamps (`ts` , `created_at` , Jira dates) into standard UTC.
- **Local Timezone Projection:** Translates UTC events into employee local timezone (9 AM–6 PM) to detect after-hours and weekend work.
- **Automated Noise Filter:** Excludes Slack bots (`bot_id`), broadcast channels (`#general` , `#announcements`), Dependabot/CI commits, and declined meetings.

3 Graceful Degradation & Role-Aware Handling (Step 5)

Missing Handles & Alternatives — FAULT TOLERANT

- **Missing GitHub Profile:** For non-engineers or unlinked profiles, the pipeline falls back to Jira velocity/throughput without crashing.
- **Slack Privacy Constraints:** If DM history is restricted, falls back to public thread activity or role-specific median baselines.

Role-Specific Baseline Adaptations — ROLE-AWARE

- **Engineers:** Focus blocks protected (24h meeting max, 12h latency tolerance).
- **Designers:** Higher iterative revision tolerance (up to 40% rework).
- **Salespeople:** Fast responsiveness required (6h latency limit, 30h meeting max).
- **Managers:** Higher meeting load baseline (34h max) and coordination focus.

EngageIQ Data Integration Pipeline

Standardized Output Schema, PACE Fusion & Database Architecture

Database Engine: Relational SQLite (SQLAlchemy 2.0)
PACE Formula: (0.40 × EHS) + (0.60 × OPI)

4 Standardized Unified Output Schema (Production Output Format)

Produced per employee per aggregation period by `MetricsAggregator.aggregate_for_employee()`:

```
{
  "employee_id": "EMP-9042",
  "employee_name": "Sarah Connor",
  "role": "Senior Backend Engineer",
  "team_id": "TEAM-ENG-03",
  "team_name": "Core Platform",
  "period_start": "2026-10-19T00:00:00Z",
  "period_end": "2026-10-23T23:59:59Z",
  "metrics": {
    "meeting_load": 0.28,                // Google Calendar: accepted meeting hours / 40h work week
    "focus_gap": 2.5,                    // Google Calendar: average unbroken focus blocks (>= 2 hours)
    "response_latency_minutes": 14.2,    // Slack: average direct mention-to-reply duration in minutes
    "after_hours_ratio": 0.08,           // Multi-Source: activity events outside local 9am-6pm / weekends
    "completion_rate": 0.91,             // Jira: resolved tickets / total assigned tickets in period
    "velocity": 13.0,                    // Jira: total story points delivered (customfield_10002)
    "deadline_adherence": 0.95,          // Jira: on-time milestone delivery rate
    "code_activity": 84.5,               // GitHub: weighted sum (1.0*Commits + 3.0*PRs + 2.0*Reviews)
    "rework_ratio": 0.04                 // Jira reopen rate + GitHub reverted commits
  }
}
```

5 Dual-Lens Scoring Engine & 40:60 PACE Fusion

Scoring Formulas (scoring/pace_engine.py)

DUAL-LENS

```
// 1. Employee Health Score (Signal Layer - 40%)
EHS = 1.0 - (0.30*meeting_load + 0.30*focus_gap +
0.20*latency + 0.20*after_hours)
// 2. Output Productivity Index (Output Layer - 60%)
OPI = (0.25*comp + 0.25*vel + 0.20*deadline +
0.20*activity) - 0.10*rework
// 3. 40:60 PACE Fusion Formula
PACE = (0.40 * EHS) + (0.60 * OPI)
```

Risk Categorization & Burnout Alert

PATTERN DETECTOR

- 🟢 **Healthy:** PACE ≥ 0.70 | 🟡 **Moderate:** 0.40 ≤ PACE < 0.70 | 🔴 **At-Risk:** PACE < 0.40.
- **collab-down-output-flat:** Flags high output (OPI ≥ 0.70) accompanied by declining collaboration health (EHS < 0.60) — critical early warning indicator before delivery collapse.
- **both-layers-down:** Simultaneous decline in health and output.

6 Relational Database Architecture (SQLite via SQLAlchemy 2.0)

Implemented in `database/models.py` with 7 relational tables matching the EngageIQ master technical document:

Table Name	Key Columns	Purpose & Relationship Mapping
`employees`	`id (PK), name, role, team_id, team_name, status, created_at`	Core employee master records linked to organizational squads.
`signal_metrics`	`id, employee_id (FK), timestamp, meeting_load, focus_gap, latency, ehs`	Persists all signal features and raw hours for auditability.
`output_metrics`	`id, employee_id (FK), timestamp, completion, velocity, rework, opi`	Persists delivery milestones, story points, and rework penalties.
`pace_scores`	`id, employee_id (FK), timestamp, ehs_score, opi_score, pace_score, risk`	Calculated PACE scores, risk bands, and explanations.
`risk_events`	`id, employee_id (FK), detected_at, risk_type, severity, evidence, status`	Stores detected burnout patterns for manager review queues.
`manager_reviews`	`id, employee_id (FK), reviewer_id, review_date, outcome, note, flag_week`	Human-in-the-loop contextual reviews and signal overrides.
`pipeline_runs`	`id, run_id (Unique), source_file, total_records, status, started_at`	Audit trail of execution history and processing metrics.

EngageIQ Data Integration Pipeline

FastAPI Endpoints, React Dashboard Integration & Test Suite

Server Port: 8000 (FastAPI + Uvicorn)
Frontend: React 18 + Vite (Port 5173)

7 FastAPI REST API & Webhook Router Matrix

Implemented in `api/server.py` and `integration/webhook_router.py`:

Method	Endpoint Path	Target Component	Operational Function
POST	`/api/integration/webhooks/slack`	Slack Events API	Captures `message` and `app_mention` events.
POST	`/api/integration/webhooks/github`	GitHub Webhooks	Captures `pull_request`, `push`, and `pull_request_review`.
POST	`/api/integration/webhooks/jira`	Jira Webhooks	Captures `jira:issue_updated` for state and changelog changes.
POST	`/api/integration/sync/google-calendar`	Google Calendar Connector	Triggers batch synchronization of meeting blocks.
POST	`/api/integration/run-aggregation`	Metrics Aggregator	Aggregates all 4 sources, computes PACE, and updates DB.
GET	`/api/scores`	React Frontend (`client.ts`)	Returns live score array consumed by `mock.applyRealData()`.
GET	`/api/organization`	React Frontend (`OrgView.tsx`)	Returns headcount, average PACE, risk distribution, and teams.
GET	`/api/teams/{team_id}`	React Frontend (`TeamView.tsx`)	Returns team member scores, sparklines, and review queues.
GET	`/api/employees/{employee_id}`	React Frontend (`EmployeeView.tsx`)	Returns 12-week history, signal/output breakdowns, and assessment.
POST	`/api/employees/{employee_id}/reviews`	React Frontend (`ReviewPanel.tsx`)	Stores contextual manager reviews directly into SQLite.
POST	`/api/upload`	React Frontend (`UploadModal.tsx`)	Accepts CSV uploads and runs the full ingestion pipeline.

8 React Dashboard Connection & Live Data Binding

How the React Frontend Connects

- The frontend in `engageiq-frontend` expects the API at `http://localhost:8000`.
- In `src/api/client.ts`, `fetchRealAndApply()` queries `/api/scores` on load.
- `mock.applyRealData(scores)` dynamically overrides mock profiles with calculated backend metrics.
- The `UploadModal.tsx` component sends CSV files directly to `POST /api/upload`.

Test Suite & Verification (19/19 Passing)

- **Unit Tests:** Validates `DataValidator`, `DataCleaner`, and `RoleAwareNormalizer`.
- **Scoring Tests:** Verifies linear EHS, OPI, and 40:60 PACE formulas and bounds.
- **Connector Tests:** Validates Calendar, Slack, Jira, and GitHub metric parsing.
- **Contract Tests:** `test_frontend_connection.py` verifies all 7 React API contracts.

9 Operational Commands

```
# 1. Run Multi-Source Integration CLI (aggregates Calendar+Slack+Jira+GitHub -> PACE):
cd c:\Users\DELL\Downloads\EngageIQ-main\EngageIQ-main\data_pipeline
python run_integration.py --employee EMP-9042

# 2. Launch FastAPI Server on port 8000:
python -m uvicorn api.server:app --host 0.0.0.0 --port 8000 --reload

# 3. Launch React Dashboard:
cd ../engageiq-frontend && npm run dev

# 4. Run Pytest Test Suite (All 19 Tests Passing):
python -m pytest tests/ -v && python test_frontend_connection.py
```